GSoC 2026 Evaluation Task — HEP-CCE

PyTorch DataLoader Microbenchmark & Darshan I/O Analysis

Author: Rohit S

1. Phase 1–3: DataLoader Benchmark Summary

Dataset & Setup (Phase 1 & 2)

Ten **.npz** shard files (502 MB each) were generated using NumPy with seed=42, totaling around 5 GB across 35,000 samples of shape (224, 224, 3). A custom Dataset class loads shards lazily and caches each shard per worker, so the same file isn't read from disk twice in a row. Images are normalised to float32 [0,1] tensors. The DataLoader runs with batch size=64, persistent workers, and prefetch_factor=2.

Results (Phase 3) — PyTorch 2.10.0, CPU-only, Windows 11, NVMe SSD:

Workers	Epoch Time (s)	Throughput (samples/s)	Speedup	Efficiency (%)
1	32.75	1068.7	1.00x	100.0
2	18.21	1921.9	1.80x	89.9
4	13.85	2526.5	2.36x	59.1
8	23.19	1509.2	1.41x	17.7

Efficiency = (T_1 / (n × T□)) × 100

Observed Scaling trends:

- Going from **1 to 2** workers gave the biggest jump, nearly 1.8x faster because workers can load and decompress the next batch while the current one is being processed.
- Moving to **4** workers pushed this further **(2.36x)**, but efficiency started dropping as the disk began to saturate.
- At **8** workers, performance actually got worse due to much of spawning that many processes on a CPU-only machine cost more in OS scheduling and inter-process overhead than it gains in parallelism.
- **4** workers were the **consistent sweet spot**.

NOTE : When re-run with **CUDA enabled (PyTorch 2.6.0+cu124, RTX 2050)**, the sweet spot shifted to 2 workers. With pin_memory active, each worker pins RAM pages for GPU transfer; this adds overhead that outweighs gains beyond 2 workers. This shows that the right worker count needs to be tuned per hardware setup.

---

2. Darshan I/O Analysis

Five Darshan (v3.5.0) logs from a **FashionMNIST PyTorch** training job (batch_size = 64, 10 epochs, 1 MPI process) were thoroughly analysed based on the **1, 2, 4, 8, and 32** threads.

Key I/O Metrics:

Threads	Runtime (s)	Read (MiB)	Written (KiB)	I/O Perf (MiB/s)
1	244.40	52.58	226.43	126.61
2	298.64	52.58	213.37	242.81
4	209.87	52.58	211.19	507.56
8	215.18	52.58	212.52	474.52
32	228.32	52.58	212.18	473.50

POSIX: 8 files across all runs (5 read-only, 2 write-only). Read volume stays constant at 52.58 MiB, every thread reads the full dataset with no partitioning. Most file reads were tiny, just 75–90 bytes each (around 230–255 operations per run), which points to config reads rather than actual bulk data loading.

Q 1 . Does I/O scale proportionally with thread count?

Not really. Performance improved about 4 times going from 1 to 4 threads (126 to 507 MiB/s), but beyond that, it stopped improving; 8T and 32T both hover around 474 MiB/s. The 2-thread run was actually the slowest, likely because it ran on a different date under different system conditions. The overall picture is clear adding more threads helps up to a point, then the bottleneck shifts elsewhere.

Q 2 . Identified bottlenecks and their likely causes:

1. **Tiny reads dominate**: 75–90 bytes per operation means each read costs a full **syscall** regardless of how little data moves, this gets expensive at scale.
2. **All threads share the same files:** With no data partitioning, every thread opens the same 8 files. At 32+ threads, this creates a queue at the filesystem metadata level before any real data transfer even begins.
3. **Diminishing returns past 4 threads:** The 32-thread run was 9% slower than 4 threads because more threads mean more scheduling noise and cache thrashing.

Q 3 . Concrete recommendations for improving ML data loading scalability

1. **Bigger reads:** Switch to WebDataset, FFCV and LMDB formats that pack samples into large sequential chunks, replacing thousands of 80-byte reads with a few large ones.
2. **Split the data across ranks:** Each process should read only its own slice of the dataset. Right now, every rank reads everything, which doesn't scale at all on multi-node systems.
3. **Don't over-provision workers:** 4 workers was the sweet spot here, pushing to 8 or 32 affected the performance, so always profile before assuming more workers means faster loading